



**Progetto:**

# UrbanMind

**Gruppo:**

**G22**

**Titolo del documento:** **Analisi e progettazione**

## Document Info

|             |                                  |             |         |
|-------------|----------------------------------|-------------|---------|
| Doc. Name   | D3-UrbanMindAnalisiProgettazione | Doc. Number | D3 V1.8 |
| Description | Documento di design              |             |         |



# Indice

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>UrbanMind</b>                                | <b>1</b>  |
| <b>Indice</b>                                   | <b>2</b>  |
| <b>Scopo del documento</b>                      | <b>3</b>  |
| <b>1. Analisi dei Componenti</b>                | <b>4</b>  |
| 1.1. Definizione dei componenti                 | 4         |
| CMP1: Gestione Autenticazione                   | 4         |
| CMP2: Frontend Cittadino (Home Pubblica)        | 4         |
| CMP3: Frontend Operatore (Dashboard Gestionale) | 5         |
| CMP4: Gestione Segnalazioni                     | 5         |
| CMP5: Gestione Proposte                         | 5         |
| CMP6: Gestione Bilancio                         | 5         |
| CMP7: Gestione Utenti                           | 6         |
| 1.2. Diagramma dei componenti                   | 6         |
| <b>2. Diagramma delle classi</b>                | <b>7</b>  |
| 2.1 Utente                                      | 7         |
| 2.2 Operatore e Admin                           | 8         |
| 2.3 Segnalazione e StatoSegnalazione            | 9         |
| 2.4 Proposta e CicloVotazione                   | 10        |
| 2.5 Bilancio e Missione                         | 11        |
| 2.6 Diagramma delle classi complessivo          | 12        |
| <b>3. Dal class diagram alle API</b>            | <b>13</b> |



## Scopo del documento

Il presente documento riporta il design del progetto **UrbanMind** tramite l'utilizzo di un diagramma dei componenti e di un diagramma delle classi. L'obiettivo è definire l'architettura logica del sistema, le interfacce tra i vari moduli e la struttura dei dati che supporterà le funzionalità di bilancio partecipativo e segnalazione cittadina.



# 1. Analisi dei Componenti

Nel presente capitolo viene presentata l'architettura in termini di componenti interni al sistema definiti sulla base dei requisiti analizzati in precedenza. Viene poi adottato l'uso del diagramma dei componenti per rappresentare l'interconnessione tra i vari moduli, identificando quindi le interfacce tra questi e verso i sistemi esterni, applicando una rigida separazione tra i flussi operativi del Cittadino e quelli dell'Amministrazione.

## 1.1. Definizione dei componenti

In questa sezione vengono descritti i componenti del sistema UrbanMind, identificando le entità autonome e le relative interfacce secondo la notazione UML standard. L'architettura riflette l'infrastruttura di deployment basata su Node.js, MongoDB e autenticazione tramite JSON Web Token (JWT).

### *CMP1: Gestione Autenticazione*

**Descrizione:** Questo componente centralizza la sicurezza e l'accesso alla piattaforma. Gestisce il riconoscimento dell'utente interfacciandosi con i provider di identità nazionali.

- **Interfaccia richiesta – credenziali:** Il componente consuma i dati di accesso provenienti dal componente esterno *Provider SPID/CIE* per validare l'identità.
- **Interfaccia fornita – autenticazione (JWT):** Produce e mette a disposizione dei moduli Frontend il token JWT (JSON Web Token), necessario per smistare gli utenti e proteggere le rotte. Non viene utilizzato alcun protocollo complesso come SAML.

### *CMP2: Frontend Cittadino (Home Pubblica)*

**Descrizione:** Modulo visivo (Frontend) dedicato esclusivamente all'utente base. È totalmente isolato dalla logica amministrativa per garantire il Role-Based Access Control.

- **Interfaccia richiesta:** Consuma il JWT per validare l'accesso. Per popolare le sue tre pagine principali, consuma unicamente le interfacce pubbliche: *consultazione bilancio, partecipazione proposte e invio segnalazioni*.



---

### **CMP3: Frontend Operatore (Dashboard Gestionale)**

**Descrizione:** Modulo visivo riservato allo staff comunale, dotato di menu laterale (sidebar). Nessuna delle sue pagine è condivisa con il Frontend Cittadino.

- **Interfaccia richiesta:** Consuma il JWT validando i privilegi di admin/operatore. Consuma unicamente le interfacce gestionali esposte dal backend: *upload bilanci*, *moderazione proposte*, *gestione segnalazioni* e *gestione utenti*.

### **CMP4: Gestione Segnalazioni**

**Descrizione:** Modulo di business logic che gestisce il flusso dei disservizi urbani, dalla creazione alla risoluzione.

- **Interfaccia richiesta – coordinate:** Consuma i dati spaziali (latitudine e longitudine) forniti dalla libreria open-source *Leaflet (Mappe)*, sostituendo soluzioni proprietarie.
- **Interfacce fornite:** Offre due interfacce distinte e non comunicanti: *invio segnalazioni* (per il frontend cittadino) e *gestione segnalazioni* (cambio stato, eliminazione, consumata dal frontend operatore).

### **CMP5: Gestione Proposte**

**Descrizione:** Modulo di business logic per il Bilancio Partecipativo.

- **Interfacce fornite:** Separa funzionalmente la *partecipazione proposte* (inserimento e voto, pubblica) dalla *moderazione proposte* (approvazione o rigetto, riservata alla dashboard).

### **CMP6: Gestione Bilancio**

**Descrizione:** Modulo dedicato alla trasparenza finanziaria.

- **Interfacce fornite:** Espone la *consultazione bilancio* in formato grafico e in sola lettura per la Home pubblica, ma riserva l'interfaccia di *upload bilanci* (caricamento dati CSV/Excel) esclusivamente alla Dashboard Operatore.



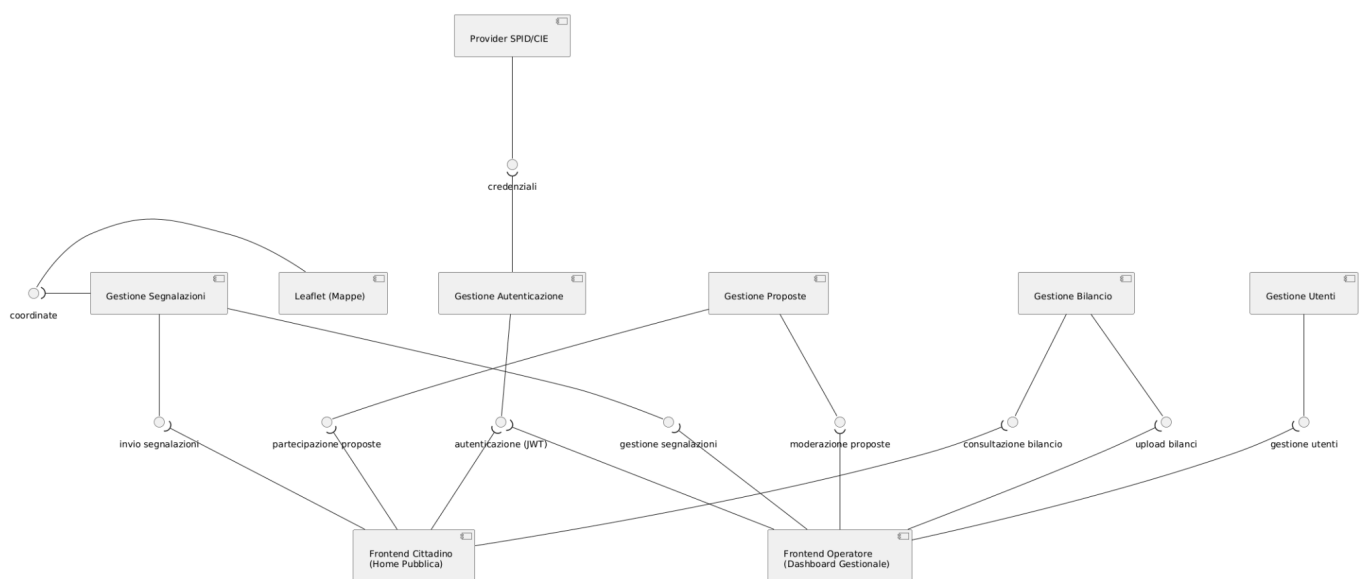
## CMP7: Gestione Utenti

**Descrizione:** Modulo di sicurezza e amministrazione ad uso esclusivo dello staff.

- **Interfaccia fornita – gestione utenti:** Mette a disposizione della Dashboard gli strumenti per visualizzare la lista account, moderare lo stato (attivo/sospeso/bannato) e modificare i permessi.

### 1.2. Diagramma dei componenti

Viene riportato di seguito un diagramma complessivo di tutti i componenti di sistema e le loro interconnessioni descritte in precedenza, utilizzando la notazione UML standard (ball-and-socket) per rappresentare le interfacce fornite e richieste.





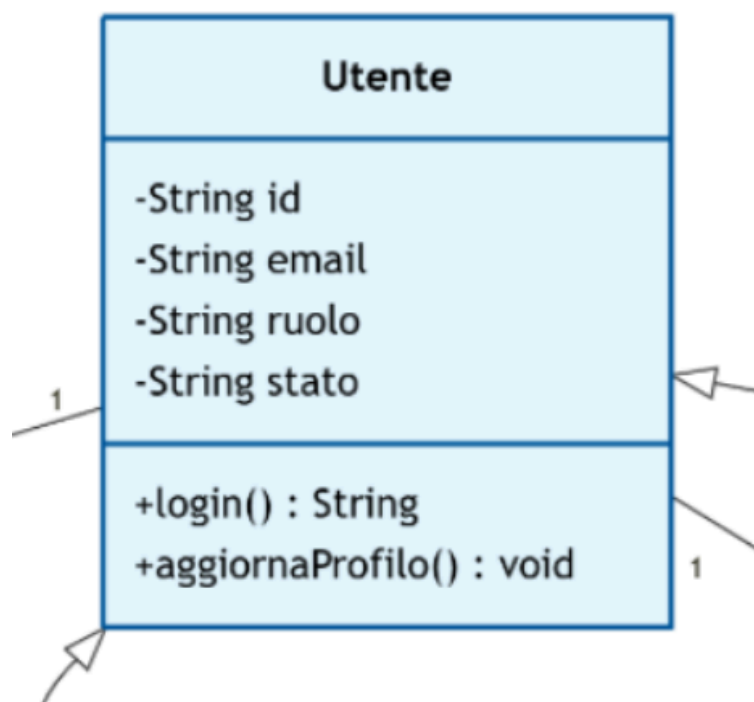
## 2. Diagramma delle classi

Nel presente capitolo vengono presentate le classi previste nell'ambito del progetto UrbanMind. Ogni componente logico del sistema è stato tradotto in una o più classi. Tutte le classi individuate sono caratterizzate da un nome, una lista di attributi (privati) che identificano i dati gestiti e una lista di metodi (pubblici) che definiscono le operazioni previste.

Per distinguere i ruoli operativi all'interno della piattaforma in base ai privilegi, sono state applicate le logiche dell'ereditarietà (Inheritance). Riportiamo di seguito le classi individuate, in questo processo si è proceduto anche nel massimizzare la coesione e minimizzare l'accoppiamento tra moduli.

### 2.1 Utente

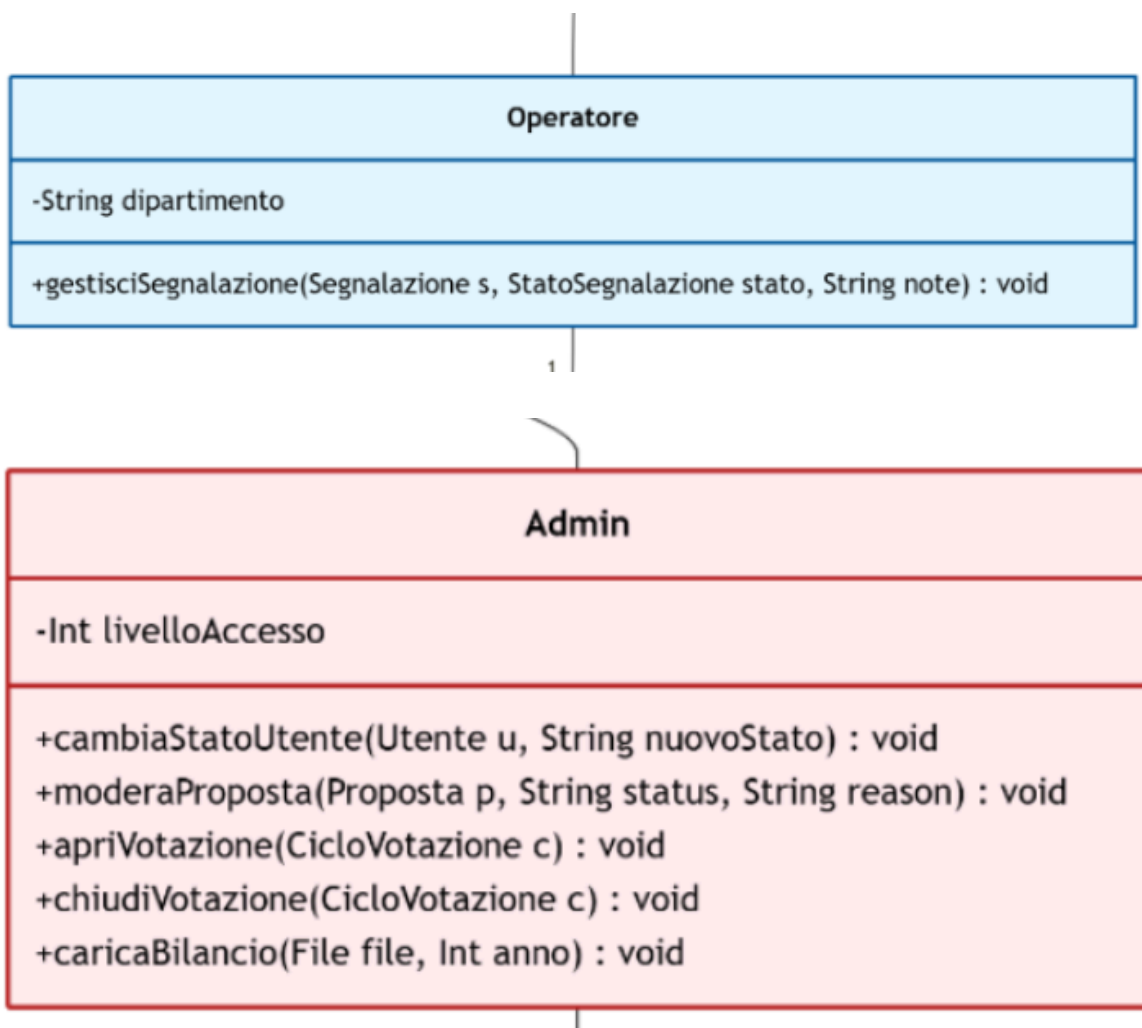
Abbiamo individuato la classe astratta principale "Utente", che rappresenta l'entità base di chiunque si interfacci con il sistema. Essa contiene gli attributi fondamentali (id, email, stato e ruolo) e i metodi per l'autenticazione. Da questa classe derivano a cascata tutti i ruoli gestionali del sistema.





## 2.2 Operatore e Admin

Queste classi ereditano le caratteristiche della classe base **Utente** (tramite relazione di *Inheritance*), ma estendono le funzionalità in base ai privilegi amministrativi assegnati. L'**Operatore** possiede l'attributo **dipartimento** e il metodo per gestire lo stato delle singole pratiche sul territorio (le segnalazioni). L'**Admin** possiede il **livelloAccesso** massimo e detiene i metodi critici per il funzionamento della piattaforma: bannare gli utenti, moderare le proposte del bilancio partecipativo, aprire o chiudere i cicli di votazione e caricare i file finanziari.

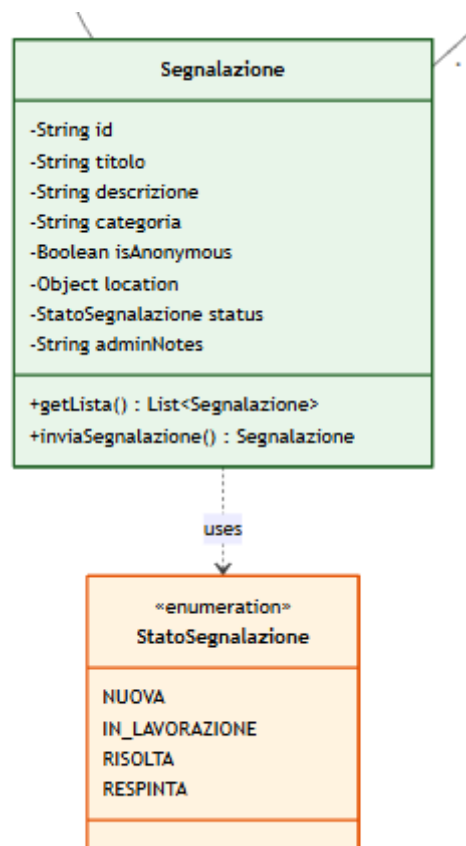






## 2.3 Segnalazione e StatoSegnalazione

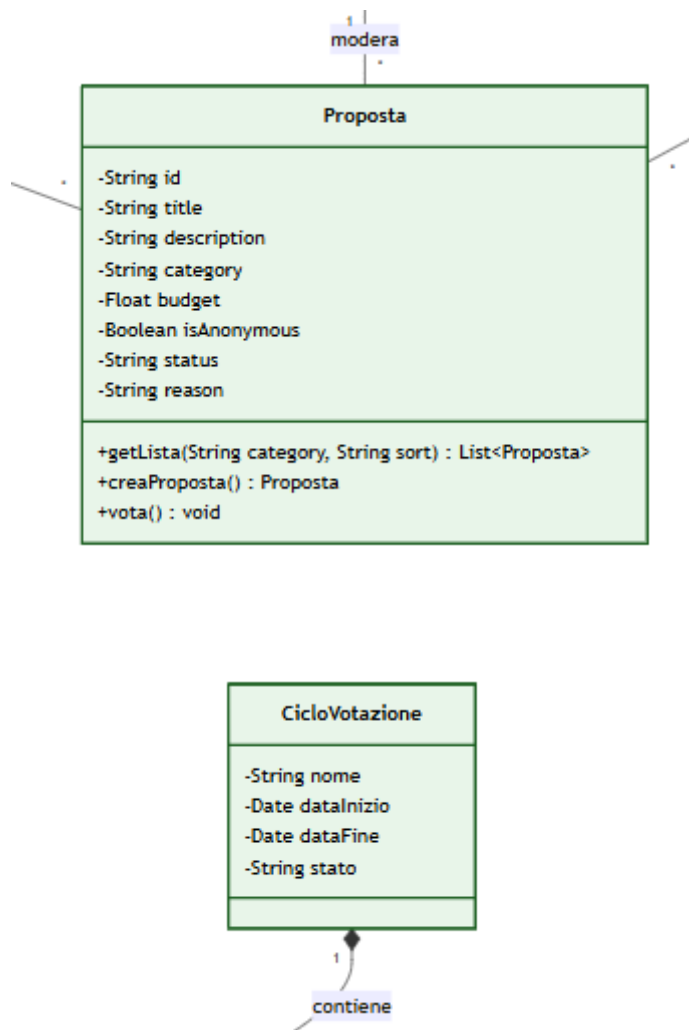
La classe **Segnalazione** racchiude tutti i dati inseriti dal cittadino quando riscontra un problema urbano, includendo un oggetto **location** per le coordinate geografiche lette dalla mappa Leaflet e il flag **isAnonymous**. Per gestire il ciclo di vita del ticket in modo tipizzato e sicuro, la classe utilizza (tramite relazione *uses*) una enumerazione chiamata **StatoSegnalazione** che ammette solo i valori predefiniti a sistema (NUOVA, IN\_LAVORAZIONE, RISOLTA, RESPINTA).





## 2.4 Proposta e CicloVotazione

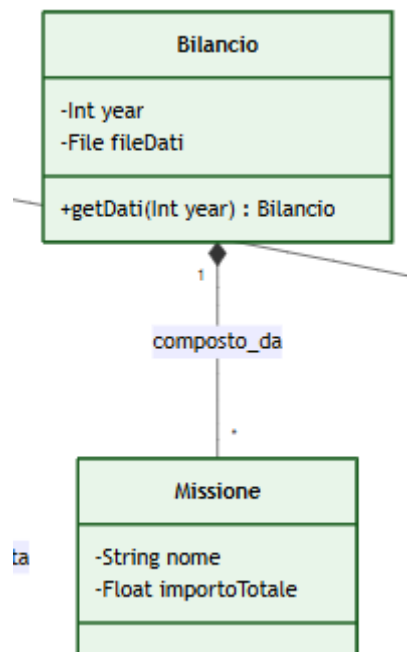
Per il dominio del Bilancio Partecipativo, sono state modellate le classi **Proposta** e **CicloVotazione**. La **Proposta** contiene l'idea del cittadino, il **budget** stimato e la **reason** (motivazione) inserita dall'Admin in fase di moderazione. Il **CicloVotazione** rappresenta la finestra temporale in cui le votazioni sono aperte; esso si relaziona alla Proposta tramite una composizione, poiché un ciclo contiene molteplici proposte ammissibili pronte per essere votate.





## 2.5 Bilancio e Missione

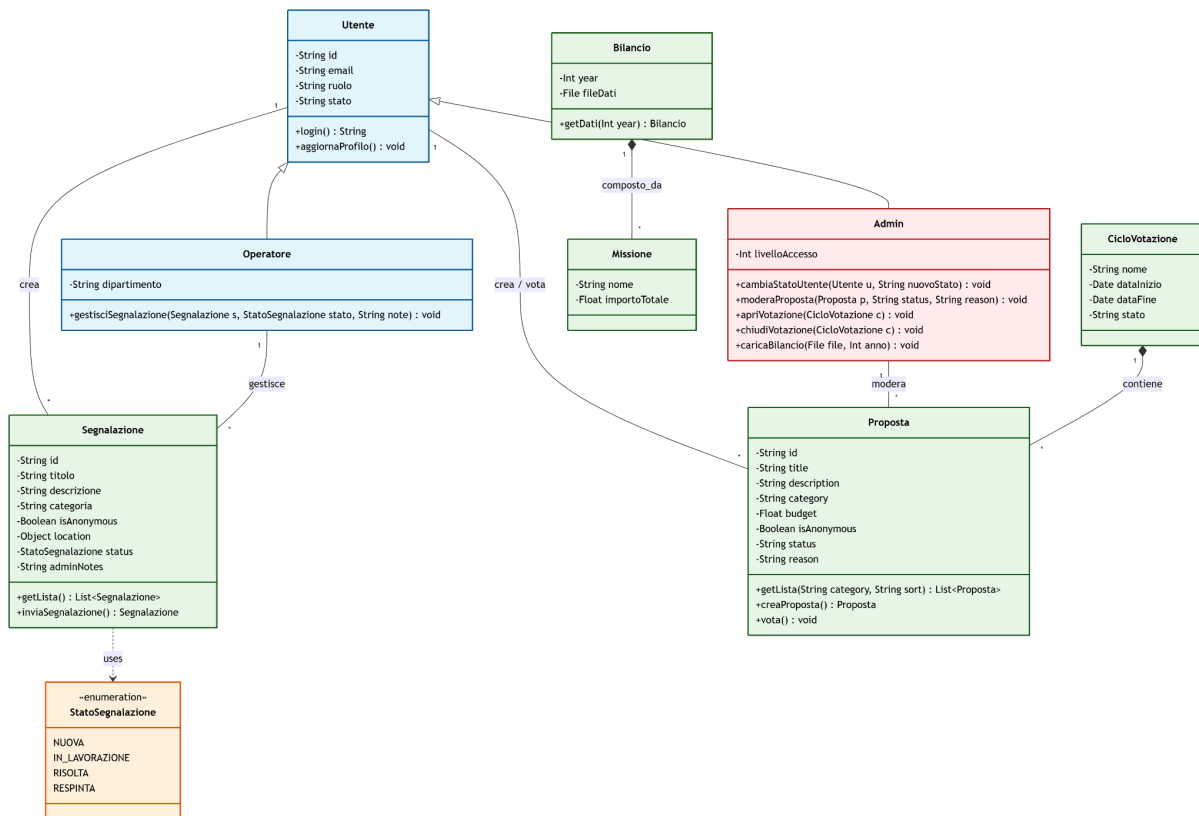
Queste classi gestiscono la trasparenza finanziaria. La classe **Bilancio** rappresenta il documento aggregato annuale e permette di esporsi al frontend tramite il metodo **getDati()**. Essendo il bilancio strutturato in più macro-voci di spesa (es. Istruzione, Viabilità), esso "è composto da" svariate istanze della classe **Missione**, che racchiude il nome della voce e l'importo totale allocato in quello specifico ambito.





## 2.6 Diagramma delle classi complessivo

Riportiamo di seguito il diagramma delle classi complessivo con tutte le entità fino ad ora presentate e le relative relazioni strutturali che intercorrono tra di esse.





### 3. Dal class diagram alle API

Riportiamo di seguito una tabella con la mappatura tra i metodi presenti nel diagramma delle classi e le API REST individuate per il backend.

| Classe       | Metodo                 | HTTP Method | URI                        | Request Body            | Response             | Note                                            |
|--------------|------------------------|-------------|----------------------------|-------------------------|----------------------|-------------------------------------------------|
| Utente       | login()                | POST        | /api/users/login           | { provider, token }     | 200 { token, role }  | Login SPID/CIE. Nessuna credenziale locale.     |
| Admin        | getListaUtenti()       | GET         | /api/users                 | -                       | 200 [ Utenti ]       | Riservato agli Admin                            |
|              | cambiaStato()          | PUT         | /api/users/:id             | { status }              | 200 Success          | Permette di sospendere o bannare un utente      |
| Segnalazione | getLista()             | GET         | /api/reports               | -                       | 200 [ Segnalazioni ] | Recupera le segnalazioni pubbliche per la mappa |
|              | inviaSegnalazione()    | POST        | /api/reports               | ReportInput             | 201 Segnalazione     | Creazione di un nuovo report urbano             |
| Operatore    | gestisciSegnalazione() | PUT         | /api/reports/{id}          | { status, adminNotes }  | 200 Segnalazione     | Modifica stato e aggiunta note                  |
| Proposta     | getLista()             | GET         | /api/proposals             | Parametri: ?cat & ?sort | 200 [ Proposte ]     | Visualizzazione idee ammissibili al voto        |
|              | creaProposta()         | POST        | /api/proposals             | ProposalInput           | 201 Proposta         | Sottomissione nuova idea per il bilancio        |
|              | vota()                 | POST        | /api/proposals/{id}/vote   | -                       | 200 Success          | Incrementa i voti. ID utente estratto dal JWT   |
| Admin        | moderaProposta()       | PUT         | /api/proposals/:id         | { status, reason }      | 200 Success          | Approvazione o rigetto con motivazione          |
|              | apriVotazione()        | POST        | /api/admin/votazioni/open  | { dataFine }            | 200 CicloVotazione   | Attiva la ricezione dei voti per il periodo     |
|              | chiudiVotazione()      | POST        | /api/admin/votazioni/close | -                       | 200 Success          | Disattiva le votazioni a fine periodo           |
| Bilancio     | getDati()              | GET         | /api/budgets               | Parametro: ?year        | 200 Dati aggregati   | Ritorna i dati per i grafici pubblici           |
| Admin        | caricaBilancio()       | POST        | /api/admin/budgets         | multipart/form-data     | 201 Success          | Upload del file CSV/Excel finanziario           |